# Assignment Code: DA-AG-010 Regression & Its Evaluation | Assignment

## Question 1: What is Simple Linear Regression?

**Answer:** **In data science, statistics, and machine learning, one of the most fundamental techniques for understanding relationships between variables is Simple Linear Regression. It is the core building block for many advanced predictive models. Despite being simple, it provides powerful insights into how one variable affects another. Businesses, scientists, and researchers use it every day to make forecasts, measure trends, and understand cause-and-effect relationships.**

**Simple Linear Regression is a method that models the relationship between two variables — one independent variable (X) and one dependent variable (Y) — by fitting a straight line through the data. This straight line represents the best possible approximation of how Y changes with respect to X.**

**The goal is straightforward:**
**To predict Y from X and understand how strongly X influences Y.**

**Before diving into the formula, assumptions, uses, advantages, and limitations, it's important to understand why this model is so widely used.**

---

**Simple Linear Regression is a statistical technique that examines the linear relationship between:**

- **Independent variable (X) → the predictor, input, or cause**
- **Dependent variable (Y) → the output, effect, or result**

**The word "simple" indicates that it uses only one independent variable.**

**The term "linear" means that the relationship between X and Y is represented by a straight line.**

**The mathematical model is:Y=a+bXY = a + bXY=a+bX Where:**

- **a (Intercept): The value of Y when X = 0**
- **b (Slope): The rate at which Y changes when X increases by one unit**
- **X: Independent variable**
- **Y: Dependent variable (predicted value)**

**This equation is called a regression line or line of best fit.**

---

## Purpose of Simple Linear Regression

**Simple Linear Regression helps us achieve three major objectives:**

**a. Prediction**

**You can predict the value of Y based on a known value of X.**
**For example:**

- **Predict *salary* based on *experience***
- **Predict *sales* based on *advertising budget***
- **Predict *temperature* based on *altitude***

**b. Understanding Relationships**

**It shows how strongly X affects Y.**
**If the slope b is large, X has a strong influence.**
**If b is near 0, X has little or no influence.**

**c. Trend Analysis**

**It helps identify patterns or trends over time.**
**For example, predicting future prices, growth trends, or demand forecasting.**

---

## The Equation Explained

**The simple linear regression equation:**

**Y=a+bXY = a + bXY=a+bX**

**Intercept (a)**

**The intercept is the point where the regression line crosses the Y-axis.**
**It answers the question:**
**"What is the value of Y when X = 0?"**

**Slope (b)**

**The slope tells us how much Y changes for one unit increase in X.**
**For example:**
**If b = 2, then every time X increases by 1, Y increases by 2.**

**Example**

**If the regression equation is:**

**Salary=25,000+4,000×(Experience)Salary = 25,000 + 4,000 \times (Experience)Salary=25,000+4,000×(Experience)**

**This means:**

- **When experience = 0 years → salary = 25,000**
- **For every additional year of experience → salary increases by 4,000**

---

## How is the Regression Line Calculated?

**The regression line is calculated using the Least Squares Method.**
**This method finds the line that minimizes the sum of squares of the errors.**

**What are errors?**

**Error (or residual) = Actual Y value – Predicted Y value.**

**The best-fitting line is the one where the total squared error is the smallest.**

**This ensures the line is as close as possible to all data points.**

---

## Assumptions of Simple Linear Regression

**To use linear regression properly, certain assumptions must hold:**

**1. Linearity**

**The relationship between X and Y must be linear.**

**2. Independence**

**Observations should be independent of each other.**

**3. Homoscedasticity**

**The variance of error terms should be the same across all levels of X.**

**4. Normality of Errors**

**Errors should follow a normal distribution.**

**5. No Outliers**

**Extreme values can distort the line.**

**Violating these assumptions can reduce model accuracy.**

---

## Visualization of Simple Linear Regression

**Imagine a scatter plot of your data:**

- **The dots represent data points.**
- **The straight line running through them is the regression line.**
- **The closeness of the dots to the line indicates how strong the relationship is.**

If the dots cluster tightly around the line → strong relationship.
If the dots are scattered widely → weak relationship.

---

## Example in Real Life

**Example: Predicting House Price Using House Size**

**Suppose you want to estimate the price of a house based on its size.**

**Your data includes:**

- **House size (X)**
- **Price (Y)**

**After applying regression, you get:**

**Price=50,000+1200×(Size in sq. ft)Price = 50,000 + 1200 \times (Size\ \text{in sq. ft})Price=50,000+1200×(Size in sq. ft)**

**Interpretation:**

- **Base price: ₹50,000**
- **Each additional square foot adds ₹1200 to the price**

**If someone wants to predict the price of a 1000 sq. ft house:**

**Price=50,000+1200×1000=12,50,000Price = 50,000 + 1200 × 1000 = 12,50,000Price=50,000+1200×1000=12,50,000**

---

## Correlation vs Regression

**Many students confuse correlation and regression.**

**Correlation**

- **Measures the strength and direction of relationship**
- **Value ranges from -1 to +1**
- **Does NOT predict**

**Regression**

- **Builds an equation to predict**
- **Shows cause-and-effect**
- **Produces a line of best fit**

**Correlation tells "how strong the relationship is",**
**Regression tells "how much Y will change if X changes".**

---

## How Do We Interpret the Slope?

**Slope (b) tells us:**

- **Direction of relationship**
- **Strength of impact**

**If b is positive → Y increases as X increases**
**If b is negative → Y decreases as X increases**
**If b is zero → No relationship**

**Example**

**If b = 0.8 (for predicting weight from height), it means:**

**"For every 1 cm increase in height, weight increases by 0.8 kg on average."**

---

## Coefficient of Determination ($R^2$)

**$R^2$ measures how well the regression line fits the data.**

**R2=Explained Variation/Total VariationR^2 = \text{Explained Variation} / \text{Total Variation}R2=Explained Variation/Total Variation**

- **$R^2 = 1$ → Perfect fit**
- **$R^2 = 0$ → No relationship**
- **$R^2 = 0.70$ → 70% of Y's variation can be explained by X**

**Higher $R^2$ means better predictive power.**

---

## Applications of Simple Linear Regression

**Simple Linear Regression is used everywhere:**

**a. Business**

- **Predicting sales**
- **Forecasting revenue**

- **Pricing analysis**
- **Budget planning**

**b. Finance**

- **Estimating stock prices**
- **Risk analysis**
- **Market trend prediction**

**c. Economics**

- **GDP forecasting**
- **Inflation modeling**
- **Population growth estimation**

**d. Healthcare**

- **Relationship between weight and blood pressure**
- **Predicting disease risks**
- **Medical cost estimation**

**e. Engineering**

- **Quality control**
- **Material strength prediction**
- **Reliability analysis**

**f. Education**

- **Predicting student performance**
- **Attendance vs grades analysis**

---

## Advantages of Simple Linear Regression

**1. Easy to understand**

**The equation is simple and interpretable.**

**2. Fast to compute**

**Works well even on small and large datasets.**

**3. Good baseline model**

**Used as a starting point before using complex models.**

**4. Works well when relationship is linear**

**Gives accurate results if assumptions hold.**

**5. Useful for prediction**

**Helps forecast future values.**

---

## Limitations of Simple Linear Regression

**1. Only one predictor variable**

**Does not handle multiple independent variables.**

**2. Assumes linearity**

**Real-world relationships may not always be linear.**

**3. Sensitive to outliers**

**Unusual data points can distort the line.**

**4. Assumes all conditions perfectly hold**

**Violating assumptions reduces accuracy.**

**5. Cannot capture complex relationships**

**Non-linear models or advanced ML algorithms may be required.**

---

## When Should You Use Simple Linear Regression?

**You should use it when:**

- **You have one independent variable**
- **You want to predict something**
- **You want to measure the effect of X on Y**
- **The relationship is roughly linear**

- You want a simple, interpretable model

It is often the first model used in analytics because it is intuitive.

---

## Example: Step-by-Step Working

Let’s understand with values:

| Experience (Years) | Salary (₹) |
|---|---|
| 1 | 25,000 |
| 2 | 30,000 |
| 3 | 32,000 |
| 4 | 40,000 |

After applying regression:

Salary=22,000+4,500×ExperienceSalary = 22,000 + 4,500 \times ExperienceSalary=22,000+4,500×Experience

Predict salary for 5 years of experience:
22,000+4,500×5=44,50022,000 + 4,500 \times 5 = 44,50022,000+4,500×5=44,500

This is how simple and powerful the model is.

---

## Residuals and Error Analysis

Residual = Actual value – Predicted value

Residuals tell us:

- How accurate the model is
- Whether assumptions are violated
- Whether the model needs improvement

If residuals show a random pattern → model is good.
If residuals show patterns → something is wrong.

---

## Overfitting and Underfitting

**Although simple linear regression rarely overfits, it can underfit.**

**Underfitting**

**Happens when the model is too simple.**
**For example, fitting a straight line to a curved relationship.**

**Overfitting**

**Less common here, but can happen with outliers.**

**The key is to ensure the relationship is truly linear.**

---

## Tools Used for Simple Linear Regression

**You can perform regression using:**

- **Microsoft Excel**
- **Python (scikit-learn, statsmodels)**
- **R programming**
- **SPSS**
- **MATLAB**
- **Google Sheets**

**In machine learning, regression models are usually created using Python.**

---

## Why Simple Linear Regression is Important

**It is the foundation for:**

- **Multiple Linear Regression**
- **Logistic Regression**
- **Time-Series Analysis**
- **Neural Networks**
- **Decision Trees**
- **Predictive analytics**

**Every data scientist must master this concept.**

---

### Summary

**Simple Linear Regression is a statistical technique used to model the relationship between one independent variable and one dependent variable using a straight line.**

**Key Points:**

- **Equation:** ***Y = a + bX***
- **Helps in prediction, trend analysis, and relationship measurement**
- **Uses Least Squares Method**
- **Must satisfy statistical assumptions**
- **Easy to interpret and widely applicable**
- **Has limitations but extremely powerful for linear relationships**

## Question 2: What are the key assumptions of Simple Linear Regression?

**Answer:The key assumptions of Simple Linear Regression ensure that the model produces reliable and accurate predictions. First, it assumes a linear relationship between the independent variable (X) and the dependent variable (Y), meaning changes in X should result in proportional changes in Y. The model also assumes independence of errors, which means the residuals (differences between actual and predicted values) should not influence each other. Another important assumption is homoscedasticity, where the variance of the errors remains constant across all levels of X; the spread of residuals should not increase or decrease as X changes. Additionally, the residuals should follow a normal distribution, which supports valid hypothesis testing and confidence intervals. Finally, the model assumes the absence of significant outliers, as extreme values can distort the slope and intercept, reducing the model's accuracy. When these assumptions are met, Simple Linear Regression performs effectively and provides meaningful insights.**

# Question 3: What is heteroscedasticity, and why is it important to address in regression models?

**Answer: Heteroscedasticity is a condition in regression analysis where the variance of the error terms (residuals) is not constant across all values of the independent variable. In an ideal regression model, one major assumption—called homoscedasticity—states that the residuals should be evenly spread out, meaning the distance between actual values and predicted values remains roughly the same no matter what the value of X is. However, when heteroscedasticity occurs, this spread becomes uneven: the error terms may be small for some values of X and large for others. This means the model's**

**prediction accuracy varies across the range of data. For example, if you are predicting income based on education, lower-income individuals may have similar incomes, while higher-income individuals may vary widely in their earnings, creating a fan-shaped or cone-shaped pattern in the residuals. Heteroscedasticity is important to identify and address because it violates a key assumption of linear regression, and this violation can lead to serious problems in the model's validity. First, heteroscedasticity does not bias the regression coefficients themselves—meaning the slope and intercept estimates remain correct on average—but it does bias the standard errors, which are crucial for hypothesis tests, confidence intervals, and p-values. When standard errors are biased, the model may incorrectly conclude that certain variables are significant when they are not, or fail to detect significance when it actually exists. This leads to unreliable hypothesis testing and misleading statistical conclusions. In practical terms, this can result in poor decision-making. For instance, a company using regression to decide how advertising affects sales might wrongly judge the effectiveness of their campaigns if heteroscedasticity distorts the model's precision. Another reason heteroscedasticity is important is that it reduces efficiency of the regression estimates. Efficiency refers to how well the model uses data to produce accurate predictions. Under heteroscedasticity, although the coefficients remain unbiased, they are no longer the "best linear unbiased estimators" (BLUE), meaning they do not achieve minimum variance. In simpler words, the model becomes less stable and less reliable, especially when predicting new values. Additionally, heteroscedasticity can make the model overly sensitive to extreme values or patterns, leading to overestimation or underestimation in certain ranges of the data. This issue becomes especially problematic in financial, economic, and risk-prediction models, where variability naturally increases with scale, such as predicting expenditure, income, stock prices, or business growth. If ignored, heteroscedasticity can cause predictive models to give overly confident or dangerously misleading forecasts. It is also important to address heteroscedasticity because it impacts model interpretability. When residual variance is inconsistent, the model may appear accurate in one part of the dataset but inaccurate in another, causing users to misinterpret how well the model truly performs. In critical fields like medicine, finance, or engineering, such inconsistencies can lead to wrong decisions or flawed strategies. Detecting heteroscedasticity usually involves visual inspection, such as plotting residuals versus predicted values; a random scatter indicates homoscedasticity, while funnel-shaped or curved patterns indicate heteroscedasticity. Statistical tests such as the Breusch–Pagan test or White's test further confirm its presence. Once identified, it must be addressed using corrective techniques. Common solutions include transforming the dependent variable (for example, using log transformation), which stabilizes the variance and reduces the impact of large errors. Another approach is using weighted least squares, which gives more importance to observations with smaller variance and less importance to those with larger variance. In machine learning applications, switching to models that naturally handle unequal variance—such as tree-based models—can also help. Alternatively, one can use robust standard errors, which adjust the standard errors without changing the regression coefficients, making hypothesis tests valid even in the presence of heteroscedasticity. Addressing heteroscedasticity is essential because the goal of**

**regression is not just to fit a line but to produce trustworthy, interpretable, and generalizable results. Without correcting heteroscedasticity, a model may appear statistically sound on the surface while producing unreliable conclusions beneath. In academic research, this can compromise the validity of findings; in business, it can lead to financial losses; and in policy-making, it can result in ineffective decisions. Therefore, understanding and correcting heteroscedasticity is a crucial step in building high-quality regression models that not only fit the data well but also provide meaningful and dependable insights.**

## Question 4: What is Multiple Linear Regression?

**Answer: Multiple Linear Regression (MLR) is an extension of Simple Linear Regression that allows us to study and model the relationship between one dependent variable (Y) and two or more independent variables ($X_1$, $X_2$, $X_3$, …). While Simple Linear Regression examines the impact of just one factor on an outcome, Multiple Linear Regression is used when the outcome is influenced by several factors simultaneously, making it a more realistic and powerful tool for prediction and analysis. The general equation for MLR is: $Y = a + b_1X_1 + b_2X_2 + b_3X_3 + \ldots + bnXn$, where *a* is the intercept (the expected value of Y when all X variables are zero), and *$b_1$, $b_2$, $b_3$*, etc., are coefficients that represent the amount of change in Y resulting from a one-unit change in the corresponding independent variable, keeping all other variables constant. This idea of "keeping other variables constant" is a major advantage of Multiple Linear Regression because it allows us to isolate the effect of each factor even when real-world variables are interrelated. For example, if we want to predict a person's salary, we might consider their years of experience ($X_1$), education level ($X_2$), skills or certifications ($X_3$), and age ($X_4$); multiple regression helps us understand how each of these factors influences salary on its own while controlling for the others. Multiple Linear Regression is widely used in fields such as economics, finance, marketing, healthcare, engineering, and social sciences because most outcomes in real life depend on several variables rather than just one. It helps answer important questions like: *Which variables significantly impact the outcome? How much does each factor contribute? How accurately can we predict future values?* Another major strength of MLR is its ability to compare the relative importance of different variables by looking at coefficient sizes, p-values, and standardized coefficients. For instance, a company trying to increase sales might study how advertising budget ($X_1$), product pricing ($X_2$), customer satisfaction ($X_3$), and store location quality ($X_4$) affect total sales (Y). The regression will not only estimate how strongly each factor influences sales but also help identify which variables the company should focus on more—perhaps advertising has a stronger effect than pricing, or location matters more than satisfaction. One of the key assumptions of Multiple Linear Regression is linearity, meaning the relationship between each independent variable and the dependent variable should be linear when the effects of other variables are controlled. Another assumption is multicollinearity should be minimal, meaning the independent variables should not be too highly correlated with each other; if they are, it becomes difficult to determine the unique contribution of each**

**variable. For example, if both "number of study hours" and "number of practice tests taken" are highly correlated, the model may struggle to identify which one truly impacts exam performance more. Other assumptions include normality of residuals, homoscedasticity (equal variance of residuals), and independence of errors, all of which ensure the regression estimates are reliable. Violating these assumptions does not always invalidate the model but can reduce accuracy and interpretability. To illustrate Multiple Linear Regression more clearly, consider a real-world example: predicting house prices. Suppose we collect data on several factors that typically influence the price of a house, such as house size ($X_1$) in square feet, number of bedrooms ($X_2$), location rating ($X_3$), age of the house ($X_4$), and distance to city center ($X_5$). After running MLR, we might obtain an equation like:**

**Price = 50,000 + 120×(Size) + 20,000×(Bedrooms) + 15,000×(Location Rating) – 1,000×(Age) – 2,500×(Distance).**

**This equation means that, holding all other variables constant, every additional square foot adds ₹120 to the price, each additional bedroom increases the price by ₹20,000, a better location rating increases price significantly, and older homes or homes farther from the city center decrease in price. Notice how Multiple Linear Regression allows us to quantify the contribution of each factor separately even though all these features jointly influence the price. With this model, we can estimate the price of any house by plugging in the values for size, bedrooms, location rating, age, and distance. For example, for a 1,000-sq-ft house with 2 bedrooms, a location rating of 4, age of 10 years, and distance of 5 km from the center, the predicted price would be:**

**Price = 50,000 + 120(1000) + 20,000(2) + 15,000(4) – 1,000(10) – 2,500(5) = Price = 50,000 + 120,000 + 40,000 + 60,000 – 10,000 – 12,500 = ₹2,47,500.**

**This example shows how MLR can be used to make accurate predictions and understand the individual effects of multiple variables interacting in complex ways. Additionally, Multiple Linear Regression helps identify which variables matter the most. If "location rating" has a large coefficient and low p-value, it may be a stronger predictor than number of bedrooms. Moreover, MLR provides measures like $R^2$ and Adjusted $R^2$ that explain how well the model fits the data; $R^2$ shows the percentage of variation in the dependent variable that is explained by all independent variables combined. A higher $R^2$ value means better prediction accuracy. Adjusted $R^2$ is especially important in MLR because it penalizes models that include too many irrelevant variables, helping prevent overfitting. Overall, Multiple Linear Regression is a powerful, flexible, and widely used statistical technique that allows researchers, students, and professionals to analyze relationships between several predictors and an outcome simultaneously. It not only helps in prediction but also in understanding the underlying structure of data, guiding decision-making, planning strategies, and evaluating the effect of multiple variables in a logical and comprehensive way.**

**Question 6: Implement a Python program to fit a Simple Linear Regression model to the following sample data: ● X = [1, 2, 3, 4, 5] ● Y = [2.1, 4.3, 6.1, 7.9, 10.2] Plot the regression line over the data points. (Include your Python code and output in the code box below.)**

Answer:--Step-by-Step Explanation of the Code

---

## 1. Importing Libraries

```
import matplotlib.pyplot as plt
import numpy np
```

### Why?

- `matplotlib.pyplot` is used to plot graphs (scatter plot and regression line).
- numpy helps with numerical operations like fitting the line.

---

## 2. Defining the Data

```
X = np.array([1, 2, 3, 4, 5])
Y = np.array([2.1, 4.3, 6.1, 7.9, 10.2])
```

### Why?

- We store the given X and Y values in NumPy arrays so we can perform mathematical operations.

---

## 3. Fitting the Simple Linear Regression Model

```
coeffs = np.polyfit(X, Y, 1)
slope, intercept = coeffs
```

### What this does:

- `np.polyfit(X, Y, 1)` fits a straight line (degree 1) to the data.
- It returns:
    - slope (m)
    - intercept (c)

- **The equation becomes:**
  **Y = mX + c**

**This is the best-fit line that minimizes the error.**

---

## 4. Predicting Y Values Using the Regression Line

```
Y_pred = slope * X + intercept
```

### Why?

- **This calculates the predicted Y values using the regression equation.**
- **These values will be used to draw the regression line.**

---

## 5. Plotting Scatter Points

```
plt.scatter(X, Y)
```

### Why?

- **A scatter plot visually shows the actual data points.**

---

## 6. Plotting the Regression Line

```
plt.plot(X, Y_pred)
```

### Why?

- **This draws the straight line that represents the fitted regression model.**

---

## 7. Adding Labels & Title

```
plt.xlabel("X")
plt.ylabel("Y")
```

```
plt.title("Simple Linear Regression Line")
```

**Why?**

- **Makes the graph easier to understand and visually clear.**

---

## 8. Displaying the Graph

```
plt.show()
```

**Why?**

- **Opens a window showing the final scatter plot + regression line.**

---

## Final Output Explanation

### ✔ Scatter plot

**Shows the actual X and Y data points.**

### ✔ Regression line

**Shows the best-fitting line according to the linear model.**

### ✔ Coefficients printed

- **Slope (m) = how much Y increases when X increases by 1.**
- **Intercept (c) = predicted Y value when X = 0.**

### Step 1: Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

---

### 🔹 Step 2: Define the Data

```
# Given data
X = np.array([1, 2, 3, 4, 5])
Y = np.array([2.1, 4.3, 6.1, 7.9, 10.2])
```

---

🔹 **Step 3: Fit Simple Linear Regression**

```
# Fit linear regression using polyfit
coeffs = np.polyfit(X, Y, 1)
slope, intercept = coeffs

print("Slope (m):", slope)
print("Intercept (c):", intercept)
```

---

🔹 **Step 4: Predict Y values**

```
# Predicted Y values
Y_pred = slope * X + intercept
```

---

🔹 **Step 5: Plot Data Points and Regression Line**

```
plt.figure(figsize=(7,5))

# Scatter plot
plt.scatter(X, Y, label="Actual Data")

# Regression line
plt.plot(X, Y_pred, label="Regression Line")

# Labels and title
plt.xlabel("X")
plt.ylabel("Y")
```

```
plt.title("Simple Linear Regression")
plt.legend()

plt.show()
```

---

🔹 **Step 6: Display Final Equation**

```
print(f"Regression Equation: Y = {slope:.2f}X +
{intercept:.2f}")
```

**Question 7: Fit a Multiple Linear Regression model on this sample data: ● Area = [1200, 1500, 1800, 2000] ● Rooms = [2, 3, 3, 4] ● Price = [250000, 300000, 320000, 370000] Check for multicollinearity using VIF and report the results. (Include your Python code and output in the code box below.)**

### Ans–

```
import numpy as np
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

# Sample data
Area = np.array([1200, 1500, 1800, 2000])
Rooms = np.array([2, 3, 3, 4])
Price = np.array([250000, 300000, 320000, 370000])

# Create DataFrame
df = pd.DataFrame({
    "Area": Area,
    "Rooms": Rooms,
    "Price": Price
})

# Independent variables
X = df[["Area", "Rooms"]]
X_with_const = sm.add_constant(X)

# Fit Multiple Linear Regression Model
model = sm.OLS(df["Price"], X_with_const).fit()

# Calculate VIF
```

```
vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

model.summary(), vif_data
```

---

Regression Model Output

The model fits extremely well ($R^2 \approx 0.999$) because the dataset is very small and clean.

VIF Results

| Feature | VIF |
|---|---|
| Area | 127.79 |
| Rooms | 127.79 |

Interpretation

- VIF > 10 indicates severe multicollinearity.
- Here, both features have VIF ≈ 128, which is extremely high.
- This means Area and Rooms are highly correlated.
- The model may produce unstable coefficients due to multicollinearity.

Multiple Linear Regression (MLR) is a statistical technique used to predict the value of a dependent variable using two or more independent variables. In this case, the dependent variable is Price, while the independent variables are Area and Rooms. The goal is to determine how changes in these predictors influence the price of a house. The dataset contains four observations, each representing a house with its area (in square feet), number of rooms, and market price.

To fit the regression model, we use the equation:

Price=β0+β1(Area)+β2(Rooms)+εPrice = β_0 + β_1(Area) + β_2(Rooms) + εPrice=β0+β1(Area)+β2(Rooms)+ε

Where:

- $\beta_0$ is the intercept
- $\beta_1$ is the coefficient for area

- $\beta_2$ is the coefficient for rooms
- ε is the error term

Using the Ordinary Least Squares (OLS) method, the Python model calculates the best-fit coefficients that minimize prediction error. The regression output shows an $R^2$ value close to 0.999, which indicates that the model explains almost all variability in house price. However, this value is misleading because the dataset is extremely small, and both independent variables are strongly correlated. The model summary also shows a high condition number (1.01e+04), which is a strong indication of potential multicollinearity problems.

To confirm multicollinearity, we compute the Variance Inflation Factor (VIF) for each predictor. VIF measures how much the variance of a regression coefficient is inflated due to multicollinearity. Its values are interpreted as follows:

- VIF < 5 → low multicollinearity
- VIF 5–10 → moderate multicollinearity
- VIF > 10 → severe multicollinearity

In the output, both Area and Rooms have a VIF value of approximately 127.79, which is extremely high and clearly indicates severe multicollinearity. This means that Area and Rooms are highly related to each other. For example, larger houses usually have more rooms; therefore, these variables provide overlapping information. Because of this, the model struggles to distinguish how much each variable independently contributes to the price.

High multicollinearity does not affect the model's overall predictive power, but it greatly affects the stability of the coefficients. It can make coefficient values fluctuate widely with even small changes in data. This results in unreliable estimates, inflated standard errors, and misleading interpretations. In our case, even though the regression model looks statistically strong (high $R^2$), the coefficients for Area and Rooms cannot be trusted.

In small datasets, multicollinearity becomes even more problematic because there is not enough variation in the data to separate the effects of the correlated variables. That is why the model produces warnings such as an invalid normality test due to fewer than eight observations.

In conclusion, while the Multiple Linear Regression model fits the given data almost perfectly, the extremely high VIF values show that multicollinearity is a serious problem. For a reliable model, more data should be collected and one of the correlated predictors (usually Rooms) may need to be removed or transformed to reduce multicollinearity.

**Question 8: Implement polynomial regression on the following data: ● X = [1, 2, 3, 4, 5] 3 ● Y = [2.2, 4.8, 7.5, 11.2, 14.7] Fit a 2nd-degree polynomial and plot the resulting curve. (Include your Python code and output in the code box below.)**

Ans– Polynomial Regression is an extension of Simple Linear Regression that allows us to model non-linear relationships between the input variable (X) and output variable (Y). While linear regression fits a straight line, polynomial regression fits a curved line by introducing higher-degree terms such as X2X^2X2, X3X^3X3, and so on. This makes polynomial regression useful when the data shows curvature and cannot be accurately represented by a straight line.

In this problem, the given dataset consists of X-values `[1, 2, 3, 4, 5]` and corresponding Y-values `[2.2, 4.8, 7.5, 11.2, 14.7]`. Observing the Y-values, it is clear that they increase at an increasing rate, indicating a non-linear upward trend. A straight line would not capture this pattern well, so fitting a 2nd-degree polynomial (quadratic function) is appropriate.

A 2nd-degree polynomial has the form:

Y=aX2+bX+cY = aX^2 + bX + cY=aX2+bX+c

Where:

- a is the coefficient of the quadratic term
- b is the coefficient of the linear term
- c is the intercept

Using NumPy's `polyfit()` function, we compute the best values for a, b, and c that minimize the error between predicted and actual values. The resulting polynomial equation (based on the tool output) is approximately:

Y=0.20X2+1.94X+0.06Y = 0.20X^2 + 1.94X + 0.06Y=0.20X2+1.94X+0.06

This quadratic curve fits the dataset significantly better than a straight line because it captures the increasing rate of change in Y as X grows. After computing the polynomial, we generate a smooth set of X values (`linspace`) to produce a continuous curve, and then evaluate the polynomial on these values to plot the final curve.

The plotted chart includes:

- Scatter points for actual data (X, Y)
- A smooth polynomial curve showing the quadratic relationship

This visualization helps to understand how the model captures the non-linearity in the data.

Polynomial Regression is very flexible, but it must be used carefully. Increasing the degree too much can lead to overfitting, where the model fits noise rather than the underlying pattern. A 2nd-degree polynomial is ideal here because the curvature is mild and the dataset small. Another important aspect is interpretability: higher-degree equations become harder to explain compared to linear models.

In summary, Polynomial Regression allows us to model curved patterns effectively using simple mathematical extensions of linear regression. In this task, the 2nd-degree model produces a smooth and accurate fit to the data, illustrating the power of polynomial methods for non-linear trends.

**Question 9: Create a residuals plot for a regression model trained on this data: ● X = [10, 20, 30, 40, 50] ● Y = [15, 35, 40, 50, 65] Assess heteroscedasticity by examining the spread of residuals. (Include your Python code and output in the code box below.)**

Answer– import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

# 1. Define the data

X_data = np.array([10, 20, 30, 40, 50])

Y_data = np.array([15, 35, 40, 50, 65])

# Reshape X for sklearn (must be a 2D array: (n_samples, n_features))

X = X_data.reshape(-1, 1)

# 2. Fit a Linear Regression model

model = LinearRegression()

```
model.fit(X, Y_data)


# 3. Calculate the predicted Y values (Y_hat)

Y_hat = model.predict(X)


# 4. Calculate the residuals (e = Y - Y_hat)

residuals = Y_data - Y_hat


# 5. Create the residuals plot

plt.figure(figsize=(10, 6))

# Plot residuals against the independent variable X_data

plt.scatter(X_data, residuals, color='blue')

# Add a horizontal line at residual = 0

plt.axhline(y=0, color='r', linestyle='--')

plt.title('Residuals Plot (Residuals vs. X)')

plt.xlabel('Independent Variable (X)')

plt.ylabel('Residuals (Y - $\\hat{Y}$)')

plt.grid(True)


# Save the plot

residuals_plot_filename = 'residuals_plot.png'

plt.savefig(residuals_plot_filename)

plt.close()
```

```
print(f"Intercept: {model.intercept_:.2f}")

print(f"Coefficient (Slope): {model.coef_[0]:.2f}")

print(f"X values: {X_data}")

print(f"Y values: {Y_data}")

print(f"Predicted Y (Y_hat): {Y_hat.round(2)}")

print(f"Residuals: {residuals.round(2)}")
```

**Code output**

```
Intercept: 6.50

Coefficient (Slope): 1.15

X values: [10 20 30 40 50]

Y values: [15 35 40 50 65]

Predicted Y (Y_hat): [18.  29.5 41.  52.5 64. ]

Residuals: [-3.   5.5 -1.  -2.5  1. ]
```

**The following steps were taken to create the residuals plot and assess for heteroscedasticity.**

## 1. Model Fitting and Residual Calculation

**A simple linear regression model was trained using the provided data: $X = [10, 20, 30, 40, 50]$ and $Y = [15, 35, 40, 50, 65]$.**

**The linear regression equation is $\hat{Y} = \beta_0 + \beta_1 X$, where $\hat{Y}$ is the predicted value of $Y$.**

| Parameter | Value |
|---|---|
| Intercept ($\beta_0$) | $6.50$ |
| Coefficient ($\beta_1$) | $1.15$ |

**The model is:**

$$\hat{Y} = 6.50 + 1.15 \cdot X$$

**Using this model, the predicted values ($\hat{Y}$) and the residuals ($e = Y - \hat{Y}$) were calculated:**

| X | Y | Predicted Y^ | Residual e |
|---|---|---|---|
| $10$ | $15$ | $18.00$ | $-3.00$ |
| $20$ | $35$ | $29.50$ | $5.50$ |
| $30$ | $40$ | $41.00$ | $-1.00$ |
| $40$ | $50$ | $52.50$ | $-2.50$ |
| $50$ | $65$ | $64.00$ | $1.00$ |

## 2. Residuals Plot

**The residuals plot displays the calculated residuals ($e$) on the vertical axis against the independent variable ($X$) on the horizontal axis. A horizontal dashed line is included at $e=0$ to serve as a reference.**

**The generated plot is shown above.**

## 3. Assessment of Heteroscedasticity

**Homoscedasticity (the ideal condition) means that the variance of the residuals is constant across all levels of the independent variable ($X$). On a residuals plot, this appears as a random, uniform band of points scattered around the $y=0$ line.**

**Heteroscedasticity (or non-constant variance) means the variance of the residuals changes as $X$ changes. This is indicated by a pattern in the spread of the residuals, such as a fan shape (increasing or decreasing variance) or a funnel shape.**

**Assessment:**

**Upon examining the residuals plot:**

- **The data points are scattered around the horizontal line at $0$.**
- **There are only five data points, which makes a definitive visual assessment difficult.**
- **However, the magnitude of the residuals does not exhibit a clear, systematic pattern of change (i.e., they don't consistently get larger or smaller) as the value of $X$ increases from $10$ to $50$. The largest residual is at $X=20$ ($e=5.5$), while the residuals at the largest $X$ values ($40$ and $50$) are smaller in magnitude ($-2.5$ and $1.0$).**

**Conclusion:**

**Based on the small number of data points and the visual distribution, there is no strong visual evidence of heteroscedasticity. The residuals appear to be relatively randomly scattered, suggesting that the assumption of homoscedasticity (constant variance of errors) is plausible for this particular model and dataset.**

---

## Python Code and Output

**The following code was used to fit the model, calculate the residuals, and generate the plot.**

**Python**

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression



# 1. Define the data
```

```
X_data = np.array([10, 20, 30, 40, 50])

Y_data = np.array([15, 35, 40, 50, 65])


# Reshape X for sklearn (must be a 2D array: (n_samples, n_features))

X = X_data.reshape(-1, 1)


# 2. Fit a Linear Regression model

model = LinearRegression()

model.fit(X, Y_data)


# 3. Calculate the predicted Y values (Y_hat)

Y_hat = model.predict(X)


# 4. Calculate the residuals (e = Y - Y_hat)

residuals = Y_data - Y_hat


# 5. Create the residuals plot

plt.figure(figsize=(10, 6))

# Plot residuals against the independent variable X_data

plt.scatter(X_data, residuals, color='blue')

# Add a horizontal line at residual = 0

plt.axhline(y=0, color='r', linestyle='--')

plt.title('Residuals Plot (Residuals vs. X)')

plt.xlabel('Independent Variable (X)')
```

```
plt.ylabel('Residuals (Y - $\\hat{Y}$)')
plt.grid(True)

# Save the plot
residuals_plot_filename = 'residuals_plot.png'
plt.savefig(residuals_plot_filename)
plt.close()

print(f"Intercept: {model.intercept_:.2f}")
print(f"Coefficient (Slope): {model.coef_[0]:.2f}")
print(f"X values: {X_data}")
print(f"Y values: {Y_data}")
print(f"Predicted Y (Y_hat): {Y_hat.round(2)}")
print(f"Residuals: {residuals.round(2)}")
```

**Output:**

```
Intercept: 6.50
Coefficient (Slope): 1.15
X values: [10 20 30 40 50]
Y values: [15 35 40 50 65]
Predicted Y (Y_hat): [18.  29.5 41.  52.5 64. ]
Residuals: [-3.   5.5 -1.  -2.5  1. ]
```

**Question 10: Imagine you are a data scientist working for a real estate company. You need to predict house prices using features like area, number of rooms, and location.**

**However, you detect heteroscedasticity and multicollinearity in your regression model. Explain the steps you would take to address these issues and ensure a robust model.**

**Answer–** **As a data scientist predicting house prices using features such as area, number of rooms, and location, the detection of both heteroscedasticity and multicollinearity signals that the current OLS model is compromised, requiring immediate remedial action to produce reliable coefficients, accurate standard errors, and valid statistical inferences, where the process of model robustification begins with tackling Heteroscedasticity, which is the condition where the variance of the residuals (the prediction errors) is not constant across all levels of the independent variables, meaning the model's errors might be small for small, cheap houses and large for large, expensive houses, and the immediate impact is that while the coefficient estimates remain unbiased, the standard errors become biased and inconsistent, leading to unreliable t-statistics, p-values, and confidence intervals, which means we cannot trust the significance tests for our features; to address this, the first and often most effective step is Data Transformation, typically involving applying a natural logarithm (ln) to the dependent variable, House Price, which helps stabilize the variance, often simultaneously correcting for issues like non-normality and non-linearity, turning the model into a log-linear model where the coefficients are interpreted as the percentage change in price for a unit change in a predictor, and if the variance is specifically related to a single predictor like Area, transforming that independent variable (e.g., using $\ln(\text{Area})$ or $\sqrt{\text{Area}}$) can also help stabilize the residual spread ; if transformations fail or if a linear interpretation is paramount, the next step is to employ Robust Standard Errors (e.g., White's standard errors or the more robust HC3/HC4 estimators), which adjust the standard error calculation to be consistent and valid even when heteroscedasticity is present, thereby correcting the inferential statistics (p-values, confidence intervals) without altering the OLS coefficient estimates themselves, providing a practical fix for inference; a more advanced technique is Weighted Least Squares (WLS), which is used when the specific functional form of the heteroscedasticity (the variance function) is known, where WLS downweights observations that have higher expected error variance and upweights observations with lower variance, giving more influence to the most reliable data points, and by correctly specifying the weights (e.g., inversely proportional to the variance function), WLS can restore the efficiency of the coefficient estimates, making them the Best Linear Unbiased Estimators (BLUE).**
**Once heteroscedasticity is addressed, attention must turn to Multicollinearity, which is the phenomenon where two or more independent variables are highly linearly correlated, such as the inevitable correlation between "Total Square Footage" and "Number of Bedrooms," or "Age of House" and "Renovation Status," and the primary impact of multicollinearity is the instability of the coefficient estimates, which become highly sensitive to small changes in the data, and the inflation of their standard errors, quantified by the Variance Inflation Factor (VIF), which measures how much the variance of an estimated regression coefficient is increased due to collinearity with the other predictors; the remediation process begins with the VIF Analysis, where a VIF value typically exceeding $5$ or $10$ for any predictor signals severe multicollinearity, prompting corrective action; the most straightforward solution is Feature Selection, where if two variables are highly correlated (e.g., VIF $> 10$) and essentially measure the same underlying concept, the variable with the lower theoretical importance or lower predictive power (as determined by initial simple regressions or domain knowledge) is simply removed, effectively breaking the collinear relationship; alternatively, if the correlated variables are both theoretically critical, they can be Combined into a Single Feature through feature engineering, such as creating a new variable like $\text{Area per Room} = \text{Total Area} / \text{Number of Rooms}$, which is often a better predictor of housing utility and inherently replaces the two collinear variables with a single,**

orthogonal one; for scenarios where correlation is high among a large group of variables or where removing features is undesirable due to potential loss of predictive power, Regularization Techniques such as Ridge Regression or Lasso Regression are the preferred advanced methods; Ridge Regression applies an $L_2$ penalty to the size of the coefficients, which stabilizes them and shrinks them towards zero, particularly reducing the large, unstable variances caused by multicollinearity without eliminating any variables entirely ; Lasso Regression (or $L_1$ penalty) achieves the same goal but has the added benefit of setting the coefficients of less important, correlated variables exactly to zero, effectively performing simultaneous feature selection and estimation; the data scientist would therefore systematically use VIF to identify the problematic pairs/groups, then prioritize domain knowledge to combine or eliminate variables, and finally use a penalized regression technique like Lasso or Ridge as a comprehensive solution, ensuring that the final house price prediction model boasts stable coefficients, valid standard errors, and robust predictive performance across the entire spectrum of the real estate market.

## Mathematical Definition of the Variance Inflation Factor (VIF)

The Variance Inflation Factor (VIF) is the metric used to quantify the severity of multicollinearity in an OLS regression model. It measures how much the variance of an estimated regression coefficient is "inflated" due to correlation with the other predictors.

### The Formula

For a predictor $X_k$, the VIF is defined as:

$$\text{VIF$$
Where:

- $\text{VIF}_k$ is the VIF for the $k^{th}$ predictor.
- $R_k^2$ is the coefficient of determination (R-squared) from an auxiliary regression.

### The Auxiliary Regression

To calculate the VIF for $X_k$, you must first run a separate regression where $X_k$ (the predictor of interest) is regressed on all the other independent variables in the model.

$$X_k = \alpha$$
The $R_k^2$ from this auxiliary regression indicates how much of the variation in $X_k$ is explained by the rest of the predictors.

### Interpretation

1. If $R_k^2 = 0$: This means $X_k$ is not correlated with any other predictor.
   $$\text{VIF}_k = \frac{1}{1 - 0} = 1$$
   A $\text{VIF} = 1$ indicates no multicollinearity. The variance of $\hat{\beta}_k$ is not inflated.
2. If $R_k^2$ approaches 1: This means $X_k$ is nearly perfectly predicted by the other variables (i.e., severe multicollinearity).
   $$\text{VIF}_k \to \frac{1}{1 - 1} \to \infty$$
   As $R_k^2$ gets closer to 1, the $\text{VIF}$ approaches infinity, indicating extreme instability and a massive inflation of the coefficient's variance.

Rule of Thumb: A $\text{VIF} > 5$ (or sometimes $> 10$) is generally considered an indication of severe multicollinearity requiring corrective action.

---

## 2. $L_1$ and $L_2$ Penalty Calculation Example

**Regularization methods (Lasso and Ridge) work by adding a penalty term to the standard OLS loss function (which is the Residual Sum of Squares, $\text{RSS}$). This penalty shrinks the magnitude of the coefficient estimates ($\beta$) to improve the model's stability and generalization.**

**The general objective function for penalized regression is:**

$$\text{Objective} = \text{RSS} + \text{Penalty Term}$$

**Let's use a simple example where our model has three features with the following estimated coefficients: $\hat{\beta}_1 = 2.0$, $\hat{\beta}_2 = -1.5$, and $\hat{\beta}_3 = 0.5$. The regularization strength is controlled by a tuning parameter, $\lambda$, which we will set to $0.1$ for this example.**

### A. $L_2$ Penalty (Ridge Regression)

**The $L_2$ penalty is proportional to the sum of the squares of the coefficients.**

$$\text{L2 Penalty} = \lambda \sum_{j=1}^{p} \beta_j^2$$

**Calculation Example:**

1. **Square the coefficients:**
    - **$\beta_1^2 = 2.0^2 = 4.0$**
    - **$\beta_2^2 = (-1.5)^2 = 2.25$**
    - **$\beta_3^2 = 0.5^2 = 0.25$**
2. **Sum of squares: $4.0 + 2.25 + 0.25 = 6.5$**
3. **Apply $\lambda$: $0.1 \times 6.5 = 0.65$**

- **Result: The $L_2$ penalty term added to the OLS loss is $0.65$.**
- **Effect: This penalty shrinks all coefficients toward zero, but it never forces them to be exactly zero. This is useful for dealing with multicollinearity without losing any feature completely.**

### B. $L_1$ Penalty (Lasso Regression)

**The $L_1$ penalty is proportional to the sum of the absolute values of the coefficients.**

$$\text{L1 Penalty} = \lambda \sum_{j=1}^{p} |\beta_j|$$

**Calculation Example:**

1. **Take the absolute values of the coefficients:**
    - **$|\beta_1| = |2.0| = 2.0$**
    - **$|\beta_2| = |-1.5| = 1.5$**
    - **$|\beta_3| = |0.5| = 0.5$**
2. **Sum of absolute values: $2.0 + 1.5 + 0.5 = 4.0$**
3. **Apply $\lambda$: $0.1 \times 4.0 = 0.40$**

- **Result: The $L_1$ penalty term added to the OLS loss is $0.40$.**
- **Effect: Because the $L_1$ constraint region is a diamond shape with "corners" on the axes, it tends to force the estimates of less significant coefficients (like $\beta_3$ in this case) to land exactly on zero. This performs automatic feature selection, which is a key advantage of Lasso.**